

Week 3 - Day 11b Supplementary: Binary Search

Binary search can be used in many different ways, not just to search in sorted lists. In these exercises, we will explore some different applications of binary search.

In each of the exercises, your task is to write a function that solves the problem. The function `check_encrypted` will print a readable message if you got the right answer, and otherwise it will print some strange characters.

```
# This function is for testing, do not modify
def check_encrypted(answer, secret):
    answer = str(answer)
    result = []
    offset = sum([ord(c) for c in answer])
    for i in range(len(secret)):
        result.append(chr(((offset + ord(answer[i%len(answer)])) +
ord(secret[i]) - 64)%224 + 32))
    print("".join(result))
```

Example: Integer Square Root

Given a positive integer n , find out if $\sqrt{n}$ is an integer and in that case return $\sqrt{n}$.

Sounds easy? We can use `m = int(n ** 0.5)` or `m = int(math.sqrt(n))` and then check if `m*m == n`. Run the cell below:

```
def integer_sqrt(n):
    m = int(n ** 0.5)
    if m*m == n:
        return m
    else:
        return -1 # no integer square root

print(integer_sqrt(25))
```

As you can see, this seems to work. However, try to replace 25 with 175899579799986082898060494914248911424055361. This time it doesn't work, because `** 0.5` uses floating point calculations which are imprecise. Instead, we can use binary search to find the greatest number m that satisfies $m^2 \leq n$. This number must be the answer if the answer exists. A typical implementation could look like this:

```
def precise_integer_sqrt(n):
    lo = 0
    hi = n+1
    while lo < hi-1:
        mid = (lo + hi) // 2 # halfway between lo and hi
```

```

        if mid*mid <= n:
            lo = mid
        else:
            hi = mid
    if lo*lo == n:
        return lo
    else:
        return -1

print(precise_integer_sqrt(175899579799986082898060494914248911424055361))

```

The numbers `lo` and `hi` are chosen such that `lo*lo <= n` and `hi*hi > n`, and these conditions are true throughout the whole search. In the end, `lo == hi-1`. This means that `lo` is the largest number that satisfies `lo*lo <= n`, so if the answer exists it must be `lo`.

Exercise 1: Phone number []

You want to call Senem, so you search for her phone number online. Unfortunately, Senem doesn't want to receive spam calls, so she has hidden her phone number by doing some complicated math. Senem's number can be represented by the integer n , but you only know the number $n^3 + n + 1$. Your task is to find n .

For example: If `hidden = 31`, then the answer is 3 because $3^3 + 3 + 1 = 31$.

```

# TODO: Your code here
def find_phone_number(hidden):
    pass

# Use this cell to test your solution. Do not modify.
check_encrypted(find_phone_number(4230042195989315025531698086403), "®
a[]{© §f}®¥¥f")

```

Exercise 2: Flowers []

There are n flowers that grow on a number line. The flower with index i is located on coordinate x_i . The list of coordinates `x` is sorted. Your task is to find a segment of d consecutive coordinates where as many flowers as possible grow.

Example: if `x = [1, 2, 4, 5, 8, 10]` and `d = 4`, then the answer is 3, because we can choose the coordinates 1, 2, 3, 4 and get the three first flowers. Another way to achieve the answer 3 is to choose the coordinates 2, 3, 4, 5.

```

# TODO: Your code here
def max_flowers(x, d):
    pass

```

```
# Use this cell to test your solution. Do not modify.
x = sorted([pow(1234567,i,1000000007) for i in range(1, 10**5+1)])
d = 350000000
check_encrypted(max_flowers(x, d), "PPPPPĚ2?Í;3.İ06=;10;Ě*:@>0;Í")
```

Exercise 3: Programming Pairing []

There are n students at AddisCoder, and there are m computers available. The number of computers is between $n/2$ and n . The students will work alone or in pairs. Every student has some previous experience of programming, and this amount is represented by the integer `experience[i]`. We want the students that work in pairs to have as similar experience as possible.

Your task is to divide the n students into m groups of size 1 or 2 so that the maximum value of $| \text{experience}[i] - \text{experience}[j] |$ among pairs (i, j) is as small as possible. What is the smallest such maximum difference we can achieve?

You are given the numbers n and m , and the list of n numbers `experience`.

Example: Let $n = 3$, $m = 2$, and `experience = [10, 50, 20]`. Since we only have two computers, two of the students will work as a pair and the third will work alone. The optimal way is to let the first and third student work as a pair. The difference between their experience is 10, so that is the answer.

```
# TODO: Your code here
def minmax_difference(n, m, experience):
    pass

# Use this code to test your solution. Do not modify.
import random
random.seed(2042)
n1 = 111
m1 = 56
experience1 = [random.randrange(10**9) for _ in range(n1)]
check_encrypted(minmax_difference(n1,m1,experience1), "Jhwqtÿdlrc%_
v_|$stpcgcfoÿnd sfh$`sqwdp#mr%:=ÿvl$Nsaeÿwryÿm_vdpwld
%dumawmnsbpcgcfo'}',ÿfr{ÿh_nÿwryÿxmluc#xgjbppqmepdr=")

n2 = 2*10**5
m2 = 140000
experience2 = [random.randrange(10**12) for _ in range(n2)]
check_encrypted(minmax_difference(n2,m2,experience2), "[]i±]Uª[]Z[]
i[]Y")
```

Exercise 4: Work Allocation []

There are n TAs in this year's edition of AddisCoder. Together, they have to complete m different tasks. Each TA can only work on one task. If k TAs work on the task with index i , it will be finished

in $\frac{t_i}{k}$ seconds. The TAs work simultaneously, so the time when they finish is the maximum of all the task times.

Write a function `find_minimum_time(n, m, t)` that takes as input two integers n and m and a list of real numbers $[t_0, t_1, \dots, t_{m-1}]$, and computes the minimum possible time for the TAs to finish their tasks if they are distributed optimally.

Example: `find_minimum_time(4, 3, [3, 1, 1])` is equal to 1.5. This is because two TAs can work on task 0, and one TA each can work on tasks 1 and 2. The times it takes to complete the tasks is then 1.5, 1, and 1. The maximum of these numbers is 1.5, which is the answer.

```
# TODO: Your code here
def find_minimum_time(n, m, t):
    pass

# Use this cell to test your solution. Do not modify.
import random
random.seed(2025)
n = 10**9
m = 10**5
t = [random.randint(1, 10**9) for _ in range(m)]
check_encrypted(round(find_minimum_time(n,m,t)),
"í==İB;FÑ=.15Ñ8<Ì3:=*>JÑB.-C47")
```

Congratulations 🎉